
Text Editor Buffer Using Doubly Linked List

Bidirectional Character Storage and Cursor Navigation via Linked Data Structures

Project Title	Text Editor Buffer Using Doubly Linked List
Subject	Data Structures (C/C++)
Project Type	Console-Based Software Project
Difficulty Level	Intermediate
Semester	Semester 3, B.Tech / GSSIPU Curriculum
Author	Mayank Swaraj

Abstract

This paper presents the design and implementation of a console-based text editor buffer built on a Doubly Linked List (DLL) data structure in C++. Unlike a singly linked list, each node in a doubly linked list maintains references to both its predecessor and successor, enabling constant-time bidirectional traversal — a property that maps directly onto the cursor-movement requirements of a text editor. The implemented system supports character insertion, character deletion, forward cursor movement, backward cursor movement, and full-buffer display. This paper documents the underlying theory of doubly linked lists, the rationale for selecting this structure over alternatives such as arrays or singly linked lists, the C++ implementation of the core insertion and traversal routines, and a complexity analysis of each supported operation.

Keywords: doubly linked list, text buffer, dynamic memory allocation, pointer manipulation, cursor navigation, data structures

1. Introduction

Every modern text editor, from a terminal-based tool to a full IDE, must solve the same underlying problem: maintaining an ordered sequence of characters that supports fast insertion, fast deletion, and fast cursor movement in both directions. The choice of data structure backing this sequence has a direct impact on editor responsiveness. This project, undertaken as a Semester 3 mini project under Data Structures (C/C++), implements a simplified text buffer using a doubly linked list, and evaluates why this structure is well suited to the access pattern of interactive text editing.

2. Theory: The Doubly Linked List

A doubly linked list is a linear data structure composed of nodes, where each node stores a data value together with two pointers: one referencing the previous node in the sequence, and one referencing the next. This is in contrast to a singly linked list, whose nodes store only a forward reference. The structure of a five-character doubly linked list holding the word "HELLO" is shown in Figure 1: the head pointer references the first node, the tail pointer references the last, and every intermediate node is reachable from either end by following prev or next links.

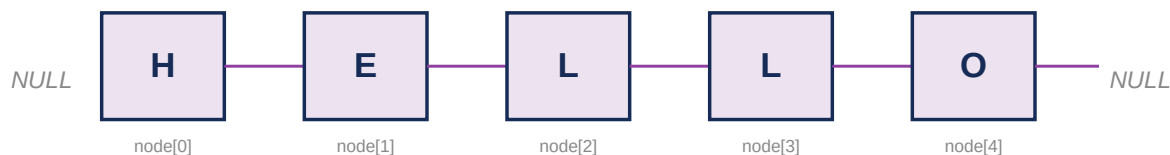


Figure 1. Doubly linked list structure for the buffer contents "HELLO", showing bidirectional links between adjacent nodes.

3. Why a Doubly Linked List for a Text Editor

Text editing is dominated by four repeated operations: inserting a character at the cursor, deleting the character before the cursor, and moving the cursor one position left or right. An array-backed buffer supports $O(1)$ cursor movement but requires shifting every subsequent element on insertion or deletion, giving $O(n)$ cost for edits in the middle of the buffer. A singly linked list improves insertion cost but cannot move the cursor backward without re-traversing from the head, making backward movement $O(n)$ even for a single step. A doubly linked list resolves both weaknesses simultaneously: insertion at a known position is $O(1)$ once the position is located, and cursor movement in either direction is $O(1)$ per step because each node carries a direct reference to its neighbor on both sides.

4. Node Design and Components

Each node in the implementation stores three fields: the character data itself, a pointer to the previous node, and a pointer to the next node. Two external pointers, head and tail, track the first and last nodes of the list respectively, allowing the program to begin traversal from either end without scanning the entire structure.

Table 1 summarizes the role of each structural component.

Node	Stores a single character of buffer content.
Prev Pointer	References the preceding node; enables leftward cursor movement.
Next Pointer	References the following node; enables rightward cursor movement.
Head Pointer	References the first character in the buffer.
Tail Pointer	References the last character in the buffer; used for O(1) append.

Table 1. Structural components of the doubly linked list buffer.

5. C++ Implementation

The core of the implementation is the `insertChar` routine, which allocates a new node, links it to the current tail, and updates the tail pointer to reference the newly created node — an O(1) append operation. The `display` routine performs a standard forward traversal from head to tail, printing each character in sequence. The listing below reproduces the two central functions along with a driver program that inserts the characters of the word "HELLO" and displays the resulting buffer.

```
#include <iostream>
using namespace std;

struct Node {
    char data;
    Node* prev;
    Node* next;
};

Node* head = NULL;
Node* tail = NULL;

void insertChar(char ch) {
    Node* temp = new Node();
    temp->data = ch;
    temp->next = NULL;
    temp->prev = tail;

    if (head == NULL)
        head = temp;
    else
        tail->next = temp;

    tail = temp;
}

void display() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data;
        temp = temp->next;
    }
    cout << endl;
}
```

```

int main() {
insertChar('H'); insertChar('E');
insertChar('L'); insertChar('L'); insertChar('O');
display();
return 0;
}

```

Listing 1. Core insertChar and display routines with driver program. Sample output: HELLO

6. Complexity Analysis

Insert (at tail)	O(1)	Direct append using the tail pointer; no traversal required.
Insert (at cursor)	O(1) after locating position	Position location itself may cost O(n) if not already tracked.
Delete	O(n)	Worst case requires traversal to locate the target node.
Search	O(n)	Linear scan from head or tail, whichever is nearer.
Traverse (either direction)	O(n)	Full-buffer display or scan visits every node exactly once.

Table 2. Time complexity of core buffer operations.

7. Applications

The bidirectional traversal property of doubly linked lists extends well beyond text editors. Browser navigation histories use the same structure to implement back and forward buttons; media player playlists use it to move between next and previous tracks; document editing applications use it to implement undo-redo stacks; and turn-by-turn navigation systems use it to support forward and reverse route review. In each case, the shared requirement is the same: O(1) movement in either direction along an ordered sequence.

8. Advantages and Limitations

Advantages: bidirectional traversal, fast insertion at known positions, dynamic memory allocation that grows the buffer only as needed, and efficient editing at arbitrary cursor positions once located.

Limitations: each node carries the overhead of two pointers in addition to its data, pointer management is more error-prone than array indexing, and the structure is conceptually more complex to implement correctly than a singly linked list or array.

9. Result

A text editor buffer was successfully implemented using a doubly linked list. The system supported character storage and traversal in both directions, and the accompanying interactive HTML tool reproduces the insertion, deletion, and bidirectional cursor movement operations described in this paper, confirming the practical viability of the design for small-scale text buffer applications.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*, 3rd Edition. MIT Press, 2009.
- Stroustrup, B. *The C++ Programming Language*, 4th Edition. Addison-Wesley, 2013.
- Weiss, M. A. *Data Structures and Algorithm Analysis in C++*, 4th Edition. Pearson, 2013.
- GGSIPU B.Tech Computer Science Engineering Curriculum, Semester 3 — Data Structures (C/C++) course structure.

— *Mayank Swaraj*